

# SDES(++)

This is my take on SDES and SDES++ tasks from San Diego CTF 2024.

## SDES

The easy SDES task is to crack a cipher with 300 encrypt queries.

You probably can just query every byte at every position to solve this, but I overcomplicated it (anyway needed for SDES++)

Let's formulate the cipher like this:

$$cipher_K = S_1 R_{K_1} \cdots S_n R_{K_n} M R_{-K_n} S_n^{-1} \cdots R_{-K_1} S_1^{-1}$$

That is, function  $cipher_K$  for some key bytes  $K_1 \cdots K_n$  is equal to a composition of functions representing the known s-boxes  $S_i$ , rotation  $R_i = (x+1)\%256$ , mirror box  $M = x^1$  and their inverses (excuse me for the unconventional notation with the functions written in the order they are applied).

Motivated by key incrementing every byte, let's increment an (almost) arbitrary key by  $\delta$  and see what is changing. Let's assume a key with  $K_1 + 16 < 256$  (so basically  $\frac{15}{16}$  of all random keys). Incrementing this key will not carry over (as we encode at most 16 bytes), so we can safely assume only  $K_1$  is incremented.

Note that  $R_{a+b} = R_a R_b$ . Here is the result:

$$cipher_{K'} = S_1 R_\delta R_{K_1} \cdots S_n R_{K_n} M R_{-K_n} S_n^{-1} \cdots R_{-K_1} R_{-\delta} S_1^{-1}$$

Let's abstract away the inner part on the right, naming that part  $B_K$ .

$$cipher_{K'} = S_1 R_\delta B_K R_{-\delta} S_1^{-1}$$

Using this equation we can encrypt/decrypt any 16 bytes using respective  $\delta$  if we know  $B_K$ . Solving for  $B_K$ :

$$B_K = R_{-\delta} S_1^{-1} cipher_{K'} S_1 R_\delta$$

Thus, if we want to know  $B_K(x)$ , we first transform  $x$  with  $R_{-\delta} S_1^{-1}$ , then plug it into the cipher and finally transform it back with  $S_1 R_\delta$ .

Notice how we now store all the info about a specific cipher instance in one byte-to-byte function. Now we can just get all 256 in input-output pairs for  $B_K$  (in  $\frac{256}{16} = 16$  queries, actually) and crack the cipher.

# SDES++

In the harder version we have two parts to the cipher: header, encrypted with a fixed key, and body, encrypted with the key contained in the header.

The interesting thing about the header is that it contains the key twice. That is the ONLY thing we know about it, besides it being a valid key decrypting the body.

Let's see what can we know about the system with only that information.

Here are two encrypting equations for equal bytes:

$$x \rightarrow S_1 R_\delta B_K R_{-\delta} S_1^{-1}$$

$$x \rightarrow S_1 R_{\delta+8} B_K R_{-\delta-8} S_1^{-1}$$

The right side (output of  $B_K$ ) we can calculate, the left side is unknown. The thing is, we know that for those two equations the value on the left of  $B_K$  differs exactly by 8! That's not much information, but is still something.

What do we get from that? From every plaintext-ciphertext byte pair, we get some outputs  $(y_1, y_2)$  corresponding to the inputs satisfying  $x_1 + 8 = x_2$ . For two pairs with a common output, we can chain them, getting  $(y_1, y_2, y_3) \rightarrow (x, x + 8, x + 16)$ . Overall, we can get 8 chains of 32 outputs each that are known to be produced from inputs with the same remainder modulo 8, and even their relative ordering.

The pairs we get every time are random, but very probable we get all of them in about 50 queries.

The variables left are the remainders themselves and which of the outputs have the least input value. That is, as expected, not much. But actually that helps to make a feasible bruteforce method.

Imagine you get a key that has only bytes that are the same modulo 8. You can relate them using the chains we built earlier and have only one unknown base byte that you can easily bruteforce. There are two problems with that:

- This has a 1 in 2 million chance of happening,
- This only helps us to decode one plaintext, for that exact session key.

To make it actually work, first of all we try getting not one, but three distinct remainders in a key. That is much more probable, although more to bruteforce. Because of that you would need to write a little faster version of the cipher (it's enough to just implement it with only constant time array access and run in pypy, cracks it in ~5 seconds).

The second problem is solved by noticing that we get all the needed info to piece together a part of the  $B_K$  table for the header cipher instance. For a given cracked key we can place the values contained in the corresponding chains at the correct place. That means we decrypt  $\frac{3}{8}$  of the cipher at a time, so a couple of tries will suffice.

There is also the fact that  $\frac{1}{16}$  have their increment carried over to other bytes, so they will not work with our scheme. Fortunately, this can be resolved with some redundancy, like choosing the most frequent from the possible byte values. For me it didn't affect anything.

The probability of hitting only 3 distinct remainders is (probably):

$$\binom{8}{3} \left(\frac{3}{8}\right)^8 \approx 1/50$$

Actually, in reality I get a bit less hit rate than this, but it's good enough anyway.

All this gets us everything needed to decrypt the header and decrypt the body directly.